

TraceSurgery: Replay-Controlled Counterfactual Interventions for Failure Causality and Recoverability in Computer-Use Agents

Aura Yavary

September 2026 — Technical Report v1.0.3

Abstract

Computer-use agents fail on long, stateful workflows for reasons that are difficult to infer from final task success or post-hoc trace inspection alone. Recent work already provides CUA-specific failure taxonomies, root-cause localization, recovery benchmarks, counterfactual repair, and minimal repair-family recovery. We therefore ask a narrower question: *when a debugger claims that a particular step or mechanism caused a failure, does an executed intervention at that checkpoint actually change the outcome?* TraceSurgery treats failed trajectories as branchable experiments. It restores evaluator checkpoints, first verifies that an unmodified suffix reproduces the source trajectory, then applies controlled action, state, or continuation interventions and re-grades the resulting world state. The resulting *counterfactual recovery surface* measures recoverability as a function of intervention time and intervention family while excluding checkpoints with poor replay fidelity from causal summaries. The repository implements the branch-and-replay engine, provenance hashes, diagnosis-faithfulness evaluation, task-level statistics, safety/state grading, and a deterministic 500-task software-validation suite. It deliberately reports no frontier-model performance: scientific claims require realistic desktop/browser environments, strong matched baselines, repeated model trials, human calibration, and task-level uncertainty.

1 Motivation and novelty boundary

Long-horizon computer use exposes agents to hidden state, delayed effects, cross-application dependencies, interruptions, and irreversible actions. OSWorld 2.0 makes this difficulty explicit with 108 realistic workflows and reports that frontier systems still lose constraints, miss changing information, guess instead of asking, and skip verification on long workflows [1]. Yet a binary success label cannot distinguish a causally critical action error, persistent environment-state corruption, stale internal state, or a still-salvageable state followed by a poor continuation policy.

Failure diagnosis and recovery are not empty research spaces. AgentErrorBench/AgentDebug study structured failure analysis [2]; AgentRx localizes critical steps [3]; CUADebug contributes a CUA-specific taxonomy, human-annotated OSWorld failures, and diagnosis-driven re-execution [4]; GUI-RobustEval/RoTS evaluates GUI recovery and synthesizes recovery-oriented trajectories at scale [5]; CausalFlow uses step-level counterfactual intervention and minimal repair [6]; and MRFR/GCJR recovers inclusion-minimal repair families with counterfactual replay [7]. Accordingly, TraceSurgery does *not* claim that taxonomies, replay, counterfactual repair, GUI recovery, or minimum repair sets are individually new.

The intended contribution is the joint experimental design: **computer-use-specific intervention decomposition over checkpoint-indexed recovery surfaces, with causal evidence gated by replay fidelity and debugger claims scored by executed intervention effect**. This is deliberately narrower than the project’s retired RecoveryBench framing.

2 Problem formulation

Let a computer-use task produce a trajectory

$$\tau = (o_0, a_0, s_0, o_1, a_1, s_1, \dots, o_T, a_T, s_T),$$

where o_t is the agent-visible observation, a_t is the action, and s_t is evaluator-only environment state. Let $G(s_T)$ be the outcome grader. For a failed source trajectory with $G(s_T) = 0$, intervention family I at checkpoint t induces a branch $\tau^{(t,I)}$ with outcome

$$R(t, I) = G(s_T^{(t,I)}).$$

The matrix of $R(t, I)$ across checkpoints and intervention families is the **counterfactual recovery surface**.

A central validity problem is replay drift. Before interpreting an intervention branch causally, TraceSurgery executes a *no-intervention control* from the same checkpoint using the recorded suffix. Let $H(\cdot)$ denote a stable hash or preregistered task-relevant state equivalence relation. Checkpoint t is causally admissible only when

$$H(s_T^{(t,\emptyset)}) = H(s_T).$$

If this control fails, intervention outcomes at t are reported but excluded from causal summaries. In stochastic GUI environments, strict equality may be replaced only by a preregistered task-relevant equivalence relation.

3 Intervention families

TraceSurgery treats interventions as evaluator-side experiments, never as evidence that the agent itself recovered.

Intervention	Surgical change	Causal question
No intervention	Replay identical suffix	Can this checkpoint reproduce the source and support causal inference?
Action patch	Replace only a_t	Was this decision causally sufficient under a fixed continuation?
Observation refresh	Regenerate o_t with world state fixed	Was stale/noisy observation the bottleneck?
State patch	Restore a reference s_t	Was accumulated external state corruption sufficient to explain failure?
Belief/memory patch	Repair exposed internal state with s_t fixed	Was internal state tracking the bottleneck?
Local replan	Replace a bounded next-action window	Can a small policy correction recover without rollback?
Suffix replan	Replace all future actions from t	Is the current world state still salvageable by a better policy?
Rollback + replan	Restore an earlier checkpoint and regenerate suffix	How much rollback is needed to regain recoverability?

The offline reference implementation executes no-intervention, action-patch, state-patch, and oracle-suffix branches. Other families are interface-level paper requirements because meaningful belief patches or replanning require access to a real agent architecture or provider harness.

4 Metrics and falsifiable diagnosis

For failed baseline outcome Y_0 and branch outcome $Y(t, I)$, the direct intervention effect is

$$\Delta(t, I) = Y(t, I) - Y_0.$$

Primary derived quantities are:

- **Replay Fidelity:** fraction of tested checkpoints whose no-intervention replay reproduces the source under the declared equivalence policy.
- **Diagnosis Faithfulness:** fraction of preregistered debugger claims for which the declared repair has positive executed outcome effect at a replay-faithful checkpoint.
- **Last Recoverable Checkpoint (LRC):** latest admissible t for which $R(t, I) = 1$ for a fixed intervention family.
- **Recovery AUC:** mean success of one intervention family over admissible normalized checkpoints.
- **Intervention-Type Decomposition:** paired contrast among action, state, internal-state, and policy surgeries.
- **Recovery Cost:** added actions, latency, token/monetary cost, and safety side effects.

The repository also implements bounded minimum-cardinality oracle action-patch search as a causal upper-bound probe. Because MRFR/GCJR already studies minimal repair-family recovery [7], this mechanism is explicitly not presented as a novelty claim.

5 Executable framework and research-engineering validity

The repository contains provider-neutral agent/environment contracts, exact pre- and post-action observations, evaluator-only snapshots, deterministic state/safety grading, perturbation events separated from agent failures, stable checkpoint/continuation/final-state hashes, replay-fidelity gates, task-clustered bootstrap analysis, and a command-line branch-and-replay engine. A deterministic local sandbox is used for CI and semantic validation rather than as a frontier capability benchmark.

A deliberately corrupted spreadsheet fixture demonstrates the semantics. The source run fails; no-intervention replay first verifies branch fidelity; patching the corrupted action at the causal step succeeds; patching an irrelevant action does not; and restoring the matched reference state after the corruption also succeeds. The test suite further evaluates a debugger claim as faithful only when the declared repair flips the executed outcome. These are software regression tests, not model-performance findings.

Each branch persists provenance sufficient for audit: source trial ID, intervention kind/checkpoint, source checkpoint hash, continuation hash, final-state hash, replay-fidelity status, grader output, and causal-admissibility status. Branches descended from one trajectory are treated as correlated observations rather than independent samples.

The release also includes an external-study handoff rather than fabricated frontier results. A deterministic, concurrency-safe run ledger expands a frozen study specification into task/model/environment/seed/repetition cells, assigns stable run identifiers, records worker claims, attempts and completion paths, and supports resumable execution. A separate scientific-evidence gate rejects local validation and synthetic demo records from performance-claim eligibility and requires explicit model revision, benchmark version, grader revision, and manifest provenance. When requested, the gate recomputes manifest SHA-256 from disk before a record can enter scientific summaries. A guarded reporting pipeline then emits machine-readable, CSV, Markdown, and LaTeX summaries only after the evidence audit passes; it fails closed rather than producing paper tables from invalid evidence. Concrete provider and realistic-environment adapters remain external integrations; their interfaces are executable contracts, not claims that those systems were run in this release.

6 Paper-scale study

The intended study uses realistic checkpointable browser/desktop tasks, preferably established environments rather than a new toy GUI world. Each failed source trajectory produces a matched branch family. The experimental unit is the *source task/trajectory*; task-level paired inference is used for model and intervention contrasts.

Required comparisons include trace-only root-cause localization, CUADebug-style diagnosis/re-execution, GUI recovery baselines, verifier-guided retry/replan, and oracle upper bounds. The decisive test is not raw debugging accuracy but whether a diagnostic method identifies interventions with measured causal effect.

We preregister seven questions: (1) how often trace-localized causes are intervention-faithful; (2) whether recoverable CUA failures decompose into action, world-state, internal-state, or continuation effects; (3) how recovery decays with intervention delay; (4) whether similarly successful models have different recovery geometry; (5) which mechanisms become irreversible fastest; (6) whether belief/world-state divergence predicts repair family; and (7) how many apparent effects disappear when replay-unfaithful checkpoints are excluded.

7 Threats to validity and safety

Evaluator state must never leak into agent observations. State restoration is an evaluator intervention, not an agent capability. Every intervention family is task-scoped: a state patch that violates task semantics is invalid rather than informative. Failed surgeries are first-class negative results. Destructive actions require isolated environments and separate safety scoring. GUI/world-model environments additionally require transition-consistency audits; recent work shows plausible one-step GUI generation need not preserve task-relevant context over multi-step rollouts [8].

A publishable release requires grader audits, contamination checks, repeated trials, human calibration, pre-registered exclusions, and task-level uncertainty. No branch count may be presented as an independent sample count.

8 Conclusion

Reliable computer-use agents need more than high final scores and plausible post-hoc explanations. TraceSurgery operationalizes a stricter standard: a causal diagnosis must survive controlled replay, and causal effect is only interpreted where the replay substrate itself is faithful. The current release establishes executable semantics, provenance, and validity gates. The remaining milestone is a realistic multi-model study showing whether recovery surfaces expose failure structure hidden by ordinary task success.

References

- [1] M. Yuan et al. *OSWorld2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks*. arXiv:2606.29537, 2026.
- [2] K. Zhu et al. *Where LLM Agents Fail and How They Can Learn From Failures*. arXiv:2509.25370, 2025.
- [3] S. Barke et al. *AgentRx: Diagnosing AI Agent Failures from Execution Trajectories*. arXiv:2602.02475, 2026.
- [4] W. Zhang et al. *CUADebug: Diagnosing and Repairing Computer-Use Agent Failures*. arXiv:2608.02643, 2026.
- [5] T. Bu et al. *Recovering Policy-Induced Errors: Benchmarking and Trajectory Synthesis for Robust GUI Agents*. arXiv:2605.29447, 2026.
- [6] A. Bonagiri et al. *CausalFlow: Causal Attribution and Counterfactual Repair for LLM Agent Failures*. arXiv:2605.25338, 2026.
- [7] B. Li et al. *Localizing Emergent Failures in Agentic AI: Recovering Minimal Repair Families via Counterfactual Replay*. arXiv:2608.29228, 2026.
- [8] L. Fu et al. *GUI-CC: Benchmarking Contextual Consistency of GUI World Models as Agent Environments*. arXiv:2609.00048, 2026.